

Relatório de Desenvolvimento – Integração Web com Banco de Dados

Sistema simples de cadastro de clientes utilizando HTML, CSS, MySQL e Python (Flask)

Matheus Becari Dias

1. Introdução

O objetivo do projeto é criar uma aplicação web capaz de receber dados de um formulário (CPF, nome, sobrenome e idade) e armazená-los em um banco de dados MySQL. Para isso, utilizou-se Python com o microframework Flask no back-end, HTML e CSS na interface, e o SGBD MySQL como repositório dos cadastros.

2. Arquivos HTML e CSS (pontos principais)

index.html

Define a estrutura da página com um formulário `<form>` que envia os dados via método POST para a rota `/cadastrar`. Utiliza campos `<input>` com atributos `name` (`cpf`, `primeiro_nome`, `sobrenome`, `idade`) que serão capturados pelo Flask. O arquivo também referencia a folha de estilo externa `style.css`.

style.css

Responsável pela apresentação visual: centraliza o formulário na tela, define cores, espaçamentos, bordas e efeitos como `focus` nos campos e `hover` no botão. A formatação torna a interface limpa e amigável, sem poluir o HTML com estilos embutidos.

3. Utilização do MySQL

O banco de dados `cadastro` foi criado previamente no MySQL, contendo uma tabela `usuarios` com as colunas `cpf`, `nome`, `sobrenome` e `idade` (todas do tipo adequado, como VARCHAR e INT).

A conexão com o banco é feita exclusivamente pelo Python utilizando a biblioteca `mysql.connector`. Toda a comunicação ocorre no momento em que o formulário é

submetido: o script abre a conexão, executa um `INSERT` e fecha a conexão, seguindo boas práticas de gerenciamento de recursos.

4. Python (Flask) – principal componente da integração

O Python atua como intermediário entre o front-end e o banco de dados, gerenciando rotas, capturando dados e executando consultas SQL.

4.1. Importação de módulos e seus papéis

```
python
```

```
from flask import Flask, render_template, request, redirect, url_for
```

```
import mysql.connector
```

- **Flask**: classe principal para criar a aplicação web.
- **render_template**: função que carrega e renderiza arquivos HTML (a “ponte” com o front-end).
- **request**: objeto que permite acessar os dados enviados pelo cliente, como parâmetros de formulário (`request.form`).
- **redirect** e **url_for** (importados mas não utilizados neste trecho) servem para redirecionar o usuário para outras rotas de forma dinâmica.
- **mysql.connector**: biblioteca oficial do MySQL para Python, responsável por estabelecer conexão, executar comandos SQL e tratar erros.

4.2. Criação da instância Flask

```
python
```

```
app = Flask(__name__)
```

`Flask(\_\_name\_\_)` cria o objeto central da aplicação. O argumento `\_\_name\_\_` ajuda o Flask a localizar recursos como templates e arquivos estáticos, determinando o caminho raiz do projeto.

4.3. Dicionário de configuração do banco de dados

python

```
db_config = {  
  
    "host": "localhost",  
  
    "user": "root",  
  
    "password": "escola",  
  
    "database": "cadastro",  
  
}
```

Esse dicionário centraliza os parâmetros de conexão com o MySQL. Ao chamar ``mysql.connector.connect(db_config)``, os valores são expandidos como argumentos nomeados, facilitando a manutenção e evitando repetição de código.

4.4. Rotas com `@app.route()`

O decorador `@app.route()` mapeia URLs para funções Python. No projeto temos duas rotas:

- **Raiz** ("/):

python

```
@app.route("/")
```

```
def index():
```

```
    return render_template("index.html")
```

Exibe a página inicial com o formulário. ``render_template`` busca o arquivo ``index.html`` na pasta ``templates`` e o envia ao navegador.

- **Cadastro** ("/cadastrar"):

python

```
@app.route("/cadastrar", methods=["POST"])
```

```
def cadastrar():
```

Configurada para aceitar apenas requisições POST (definido em `methods=["POST"]`). A função associada coleta os dados do formulário através de `request.form["cpf"]`, `request.form["nome"]`, etc., insere-os no MySQL e retorna uma mensagem de sucesso ou erro.

A conexão com o banco é aberta apenas quando a rota `/cadastrar` é acessada. Um bloco `try-except` captura exceções do `mysql.connector.Error`, garantindo que falhas de conexão ou SQL sejam tratadas adequadamente. Após a inserção, o cursor e a conexão são fechados para liberar recursos.

5. Conclusão

Até o momento, o projeto já realiza a comunicação completa entre o formulário HTML e o banco de dados MySQL, mediada pelo Flask. A arquitetura é simples e modular: o HTML/CSS cuida da interface; o dicionário `db_config` facilita a configuração do banco; as rotas do Flask gerenciam as requisições; e o `mysql.connector` executa as operações de persistência. Os pontos solicitados – `Flask`, `render_template`, `request`, `mysql.connector`, `Flask(__name__)`, o dicionário de configuração e `@app.route()` – foram detalhados, demonstrando seu papel essencial na integração web com banco de dados.